Hands-On-Learning: Automate LLM Pipeline: Deploy TechCorp’s Multi-Model Infrastructure

Learning Objectives

By the end of this activity, learners will be able to:

- LO1: Build optimized Docker containers for LLM applications using multi-stage builds and layer caching.
- LO2: Create a GitHub Actions workflow for automated LLM container builds with security scanning.
- LO3: Implement blue-green deployment strategies for zero-downtime model updates.



- LO4: Define cloud infrastructure using Terraform for reproducible LLM deployments.
- LO5: Compare automated pipeline performance against manual deployment benchmarks.



Description

In this hands-on practice, you will build an automated CI/CD pipeline for TechCorp's multi-model LLM infrastructure, reducing deployment time from 6 hours to under 30 minutes while achieving zero-downtime updates.

Structure of the Activity

coursera

Title: Automating CI/CD Pipeline for LLM Infrastructure Deployment

Tools to Be Used:

- Docker Desktop or Docker Engine
- GitHub Actions (free tier sufficient)
- Terraform CLI (open source)
- AWS CLI or cloud provider CLI

coursera

Setup Requirements:

- Docker Desktop installed and running.
- GitHub account with repository access.
- Terraform CLI installed (version 1.0+).
- AWS/Azure/GCP free tier account (or local Minikube for Kubernetes).
- At least 8GB RAM for container builds.

Scenario

You are a DevOps Engineer at TechCorp, a financial services company where manual LLM deployments take 6 hours and have caused three production incidents ($200K lost revenue). Your task is to build an automated CI/CD pipeline that reduces deployment time to under 30 minutes with zero-downtime updates.

Getting Started:

1. Download and extract LLM-Deployment-Starter-Files.zip from the course resources.
2. Navigate to the Module1_Deployment folder and review M1_app.py (FastAPI application) and M1_Dockerfile.template.
3. Set up your development environment with Docker, Terraform, and cloud CLI tools.
4. Build your solution incrementally using the provided templates, testing each component before integration.

coursera

Activity 1: Create Optimized Dockerfile

Aligned LO: LO1

Steps to Complete:

Step 1: Analyze the Base Image

- Review the provided M1_Dockerfile.template and identify inefficiencies.
- Note which layers will change frequently vs. Rarely.`
- Calculate the current image size using the docker images command.

coursera

Step 2: Implement Multi-Stage Build

- Create a builder stage using NVIDIA CUDA base image with Python.
- Install build dependencies in the builder stage.
- Create a runtime stage with a minimal CUDA runtime image.
- Copy only necessary artifacts from builder to runtime.

Step 3: Optimize Layer Caching

- Order COPY commands from least to most frequently changing.
- Copy requirements.txt before application code.
- Add model weights download as a separate cached layer.

Step 4: Build and Verify

- Build the optimized image and compare the size to the original.
- Verify the container starts correctly and serves requests.
- Document the size reduction achieved (target: <20GB).

coursera

Tips:

- Use .dockerignore to exclude unnecessary files from context.
- Consider using slim or Alpine variants where GPU support isn't needed.
- Test layer caching by making a small code change and rebuilding.

Activity 2: Create GitHub Actions Workflow with Security Scanning

Aligned LOs: LO2, LO3

Steps to Complete:

Step 1: Create Workflow File Structure

- Create .github/workflows/deploy-llm.yml file.
- Define triggers for push to the main branch and model artifact updates.
- Set up environment variables for container registry credentials.

coursera

Step 2: Implement Build Stage with Security Scanning

- Add checkout and Docker login steps.
- Configure Docker layer caching using actions/cache.
- Build and tag the container image with commit SHA.
- Add Trivy security scanning step with failure threshold for HIGH/CRITICAL vulnerabilities.

coursera

Step 3: Configure Blue-Green Deployment

- Add a deployment step to push the image to the container registry.
- Configure blue-green deployment with health check validation.
- Set up automatic rollback on health check failure.
- Add a notification step for deployment status.

coursera

Step 4: Test the Pipeline

- Push a commit to trigger the workflow.
- Monitor pipeline execution in GitHub Actions UI.
- Verify security scan results and deployment completion.
- Document pipeline execution time.

Tips:

- Use GitHub secrets for all sensitive credentials.
- Add timeout limits to prevent runaway builds.
- Consider parallel jobs for faster execution.

coursera

Activity 3: Define Infrastructure with Terraform

Aligned LOs: LO4, LO5

Steps to Complete:

Step 1: Create Terraform Module Structure

- Create main.tf, variables.tf, and outputs.tf files.
- Define provider configuration for your cloud platform.
- Set up backend configuration for state management.

coursera

Step 2: Define Compute Resources

- Create EC2/Compute instance with GPU (g5.xlarge or equivalent).
- Configure instance with appropriate AMI and user data.
- Set up container registry (ECR/ACR/GCR).

Step 3: Configure Networking and Security

- Create VPC with public and private subnets.
- Configure security groups with minimum required ports.
- Set up Application Load Balancer for traffic distribution.

coursera

Step 4: Validate and Compare Performance

- Run terraform plan and review proposed resources.
- Parameterize configuration for dev/staging/prod environments.
- Document infrastructure provisioning time vs. manual setup.
- Calculate total deployment time (build + scan + deploy + provision).

coursera

Tips:

- Use terraform fmt to maintain consistent code style.
- Add resource tagging for cost tracking.
- Consider modules for reusable infrastructure components.

Compile Your Findings

Requirement 1: Optimized Docker Container

Demonstrate containerization best practices for LLM applications.

Include:

- Optimized Dockerfile with multi-stage build.
- Image size comparison (before vs. after optimization).
- Layer caching strategy documentation.
- Working on container health check verification.

coursera

Requirement 2: CI/CD Pipeline with Security Scanning

Demonstrate automated build and security scanning workflow.

Include:

- GitHub Actions workflow YAML file.
- Trivy security scan configuration and results.
- Evidence of successful pipeline execution.
- Pipeline execution time documentation.

Requirement 3: Blue-Green Deployment Configuration

Demonstrate a zero-downtime deployment strategy.

Include:

- Blue-green deployment configuration.
- Health check validation setup.
- Automatic rollback trigger configuration.
- Evidence of zero downtime during deployment test.

coursera

Requirement 4: Infrastructure as Code

Demonstrate reproducible infrastructure provisioning.

Include:

- Terraform configuration files (main.tf, variables.tf, outputs.tf).
- terraform plan output showing resources.
- Environment parameterization (dev/staging/prod).
- Resource tagging strategy.

Requirement 5: Performance Comparison Report

Demonstrate measurable improvement over manual deployment.

Include:

- Deployment time comparison (manual 6 hours vs. automated).
- Error rate comparison (manual incidents vs. automated).
- Resource utilization metrics.
- Business analysis (3-5 sentences): How much time was saved? What errors were eliminated? Is this ready for production use?

Summary

This activity provides hands-on experience with automated LLM deployment through practical implementation of CI/CD pipelines. Participants have developed competency in Docker containerization, GitHub Actions workflows, and Terraform infrastructure-as-code, which are directly applicable to enterprise MLOps environments.

Key takeaways:

- Multi-stage Docker builds and layer caching reduce image sizes by 50-70% and dramatically improve build times for LLM applications.
- Automated CI/CD pipelines with security scanning transform 6-hour manual deployments into reliable 30-minute automated workflows.
- Blue-green deployments with health checks enable zero-downtime updates, a pattern used by GitHub Copilot and similar production LLM platforms.

Resources

Primary Materials (from LLM-Deployment Repository files):

- M1_app.py - FastAPI LLM inference application with health endpoints
- M1_Dockerfile.template - Inefficient Docker build (to be optimized)
- M1_requirements.txt - Python dependencies for the application
- M1_main.tf, M1_variables.tf, M1_outputs.tf – Terraform infrastructure templates
- M1_github_workflow.yml - GitHub Actions CI/CD template with TODOs

Reference Documentation:

- Docker Documentation: https://docs.docker.com/
- GitHub Actions Documentation: https://docs.github.com/en/actions
- Terraform Documentation: https://developer.hashicorp.com/terraform

Solution/Exemplar:

- Hands_On_Practice_Module1_Solution

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.